

从注册火山引擎到接入当前 Agent 工作区流程

本文档从零开始说明如何注册火山引擎、开通火山方舟 Ark、创建 `deepseek-v4-flash` 接入点，并把它接入当前 `multiple_agent_finance` 工作区，作为所有 Agent 的统一基础 LLM。

1. 注册火山引擎账号

1. 打开火山引擎官网：

`https://www.volcengine.com/`

2. 点击注册或登录。
3. 使用手机号、邮箱或企业账号完成账号注册。
4. 登录火山引擎控制台。
5. 根据平台要求完成实名认证。

个人学习账号通常使用个人实名认证；企业项目建议使用企业认证。

2. 开通火山方舟 Ark

1. 进入火山引擎控制台。
2. 搜索或进入：

火山方舟 / Ark

3. 如果首次使用，需要点击开通服务。
4. 确认服务区域。当前项目默认使用：

`cn-beijing`

5. 开通后进入方舟控制台。

3. 创建或选择 deepseek-v4-flash 接入点

火山方舟 OpenAI-compatible API 中，`model` 字段通常不是直接填写展示模型名，而是填写控制台生成的“接入点 ID”。

操作流程：

1. 在方舟控制台进入模型或模型广场。
2. 搜索：

`deepseek-v4-flash`

3. 选择该模型。
4. 创建接入点。
5. 接入点创建完成后，复制接入点 ID。

接入点 ID 通常类似：

`ep-xxxxxxxxxxxxxxxxxx`

6. 该接入点 ID 后续写入项目 `.env` 的：

`MAF_LLM_ENDPOINT_ID=ep-xxxxxxxxxxxxxxxxxx`

4. 创建 API Key

1. 在火山引擎控制台进入 API Key 管理。
2. 创建新的 API Key。
3. 复制 API Key。
4. 不要把 API Key 写入代码、文档正文或 GitHub。
5. 当前项目只允许把真实 key 放在本地 `.env` 文件中。

`.env` 已被 `.gitignore` 忽略，不会被提交。

5. 当前项目的 LLM 接入结构

统一 LLM 工厂位于：

`src/multiple_agent_finance/llm/base.py`

统一导出入口：

```
src/multiple_agent_finance/llm/__init__.py
```

所有 Agent 后续统一通过以下方式获取基础 LLM：

```
from multiple_agent_finance.llm import get_agent_llm

llm = get_agent_llm("technical_agent")
```

当前 agent_name 是预留参数。现阶段所有 Agent 使用同一个火山方舟基础模型。

6. 配置 .env

在项目根目录创建或修改：

```
C:\Users\10136\Desktop\trading_agent_learn\.env
```

写入：

```
MAF_LLM_PROVIDER=volcengine_ark
MAF_LLM_MODEL=deepseek-v4-flash
MAF_LLM_ENDPOINT_ID=你的火山方舟接入点ID
MAF_LLM_BASE_URL=https://ark.cn-beijing.volces.com/api/v3
MAF_LLM_API_KEY=你的火山引擎API_KEY
MAF_LLM_TEMPERATURE=0.2
MAF_LLM_TIMEOUT=60
```

字段说明：

- MAF_LLM_PROVIDER：LLM 提供方标识，当前使用 volcengine_ark。
- MAF_LLM_MODEL：项目语义上的目标模型名，当前为 deepseek-v4-flash。
- MAF_LLM_ENDPOINT_ID：火山方舟控制台中的实际接入点 ID。
- MAF_LLM_BASE_URL：火山方舟 OpenAI-compatible 接口地址。
- MAF_LLM_API_KEY：火山引擎 API Key。
- MAF_LLM_TEMPERATURE：生成温度。
- MAF_LLM_TIMEOUT：请求超时时间，单位秒。

7. 模型名和接入点 ID 的区别

项目中有两个概念：

```
MAF_LLM_MODEL=deepseek-v4-flash  
MAF_LLM_ENDPOINT_ID=ep-xxxxxxxxxxxxxxxxxx
```

区别：

- MAF_LLM_MODEL 表示你希望使用的模型名称。
- MAF_LLM_ENDPOINT_ID 表示火山方舟实际可调用的接入点。

实际传给 SDK 的模型字段是：

```
settings.llm_runtime_model
```

运行规则：

如果配置了 MAF_LLM_ENDPOINT_ID -> 使用 MAF_LLM_ENDPOINT_ID
否则 -> 使用 MAF_LLM_MODEL

如果 live 调用返回：

```
InvalidEndpointOrModel.NotFound
```

通常说明你没有配置接入点 ID，或者当前账号没有该接入点/模型权限。

8. 安装和激活项目环境

进入项目目录：

```
cd C:\Users\10136\Desktop\trading_agent_learn
```

激活环境：

```
conda activate multiple_agent_finance
```

如果缺少依赖，安装：

```
pip install -r requirements.txt
```

关键依赖：

```
langchain-openai
openai
pydantic-settings
python-dotenv
```

9. 离线配置检测

运行：

```
python scripts/check_llm_connection.py
```

成功时应看到：

```
{
  "status": "ok",
  "expected_model": "deepseek-v4-flash",
  "checks": {
    "provider_configured": true,
    "model_is_deepseek_v4_flash": true,
    "runtime_model_configured": true,
    "endpoint_id_configured": true,
    "base_url_configured": true,
    "api_key_configured": true
  }
}
```

说明：

- `model_is_deepseek_v4_flash=true` 表示目标模型名正确。
- `endpoint_id_configured=true` 表示已经配置火山方舟接入点 ID。
- `api_key_configured=true` 表示当前进程已读到 API key。

10. Live 连通性检测

运行：

```
python scripts/check_llm_connection.py --live
```

脚本会发送最小请求：

```
Reply with exactly: deepseek-v4-flash connected
```

成功时应看到：

```
{
  "live_check": {
    "status": "ok",
    "response_preview": "deepseek-v4-flash connected"
  }
}
```

如果失败并返回：

```
InvalidEndpointOrModel.NotFound
```

处理方式：

1. 回到火山方舟控制台。
2. 确认 `deepseek-v4-flash` 接入点已经创建。
3. 确认 API key 对该接入点有权限。
4. 复制接入点 ID。
5. 写入 `.env`：

```
MAF_LLM_ENDPOINT_ID=你的接入点ID
```

6. 重新执行 live 检测。

11. 当前 Agent 如何接入 LLM

统一调用方式：

```
from multiple_agent_finance.llm import get_agent_llm

llm = get_agent_llm("planner")
response = llm.invoke("Say hello")
```

底层实现：

```
ChatOpenAI(
    model=settings.llm_runtime_model,
    api_key=SecretStr(settings.llm_api_key),
    base_url=settings.llm_base_url,
    temperature=settings.llm_temperature,
    timeout=settings.llm_timeout,
)
```

注意：

- 当前本周单链路仍以规则和技术指标计算为主。
- LLM 工厂已经完成接入。
- 后续可以逐步把 Planner、Decision、Reflection 的自然语言总结替换为真实 LLM 调用。

12. 当前 Graph 范围

本周主图只保留：

```
planner -> technical_agent -> decision_agent -> reflection_agent -> final_report
```

暂时暂停的节点：

```
data_collection_agent
company_agent
financial_agent
news_agent
risk_agent
```

这些节点后续恢复时，不需要重新配置模型，只需要通过：

```
get_agent_llm(agent_name)
```

获取当前统一基础 LLM。

13. 单链路验证命令

以科大讯飞 002230.SZ 为例：

```
python -m multiple_agent_finance.main `
  --mode technical-chain `
  --ticker 002230.SZ `
  --date 2026-04-15 `
  --market-period 2026-03-16_to_2026-04-15 `
  --market-data-csv data/raw/market/002230_SZ_20260316_20260415.csv
```

输出报告目录：

```
outputs/reports/
```

14. 常见问题

14.1 api_key_configured=false

原因：当前环境没有读到 MAF_LLM_API_KEY。

处理：

```
$env:MAF_LLM_API_KEY="你的API_KEY"
python scripts/check_llm_connection.py
```

或写入项目根目录 .env。

14.2 endpoint_id_configured=false

原因：没有配置 MAF_LLM_ENDPOINT_ID。

如果 live 调用能成功，可以暂时不处理。

如果 live 调用失败，应去方舟控制台复制接入点 ID，并写入：

MAF_LLM_ENDPOINT_ID=火山方舟控制台里的接入点ID

14.3 InvalidEndpointOrModel.NotFound

原因：火山方舟不接受当前 runtime model，或账号没有权限。

处理：

1. 检查 MAF_LLM_ENDPOINT_ID 是否填写正确。
2. 检查接入点是否启用。
3. 检查 API key 是否有权限访问该接入点。
4. 检查模型是否确实是 deepseek-v4-flash。

14.4 ModuleNotFoundError: langchain_openai

原因：当前 Python 环境缺少依赖。

处理：

```
conda activate multiple_agent_finance
pip install -r requirements.txt
```

14.5 PowerShell 显示中文乱码

报告和文档使用 UTF-8 写入。PowerShell Get-Content 显示乱码通常是控制台编码问题，不代表文件损坏。

可用 Python 验证：

```
python -c "from pathlib import Path; print(Path('docs/volcengine_ark_agent_llm_setup.md').read_1"
```

15. 安全要求

- 不要把真实 API key 写入代码。
- 不要把 .env 提交到 GitHub。
- 不要把 API key 写入 Markdown 文档。
- 审计日志只能记录 api_key_configured=true/false，不能记录 key 内容。
- 如果 API key 已经公开暴露，应立即在火山引擎控制台轮换。

16. 后续开发建议

建议按以下顺序逐步接入 LLM：

1. 保持 Technical Agent 的数值计算由工具层完成，不让 LLM 编造价格和指标。
2. Planner 使用 LLM 做任务解释，但输出仍保持结构化 `planner_tasks`。
3. Decision 使用 LLM 做中文摘要和理由解释，但风险分数仍由规则和结构化指标约束。
4. Reflection 使用 LLM 检查报告文本一致性，但缺失字段检查仍由代码规则执行。
5. 恢复 Company、Financial、News、Risk Agent 时，统一调用 `get_agent_llm(agent_name)`。